

Formal Problem Definition

Distributed Online Conformance Checking via Trie-Based Alignment Decomposition

1 Definitions

We begin by establishing all definitions required for the formal problem statement and the subsequent proof of soundness.

1.1 Located Activities and Traces

Definition 1 (Located activity). *Let $\mathcal{A}$ be a finite activity alphabet and $N = \{n_1, \dots, n_k\}$ a finite set of network nodes. A located activity is a pair $\ell = (a, n) \in \mathcal{A} \times N$, written $n(a)$. We define $\text{act}(\ell) = a$ and $\text{loc}(\ell) = n$.*

Old

Flaw: Activity naming with occurrence counters. Currently, we construct activity labels as $a' = \text{name-location-occurrence}$ (e.g. CRP-B0, CRP-B1), embedding the occurrence index into the activity name. Formally, this replaced $\mathcal{A}$ with a much larger alphabet $\mathcal{A}' = \mathcal{A} \times N \times \mathbb{N}$, making the trie hyper-specific to training data and destroying generalisation.

New

Fix: Canonical activity names. Activities use the original name only: $a \in \mathcal{A}$. The located activity (a, n) separates identity from location.

Definition 2 (Located activity equality). *Two located activities $\ell_1 = (a_1, n_1)$ and $\ell_2 = (a_2, n_2)$ are equal, written $\ell_1 = \ell_2$, if and only if $a_1 = a_2$ **and** $n_1 = n_2$.*

A weaker predicate $\text{sameActivity}(\ell_1, \ell_2)$ holds iff $a_1 = a_2$ (ignoring location).

Old

Flaw: Location-ignoring equality. The original equality checked only $a_1 = a_2$, ignoring the location component. This caused incorrect trie matches: an activity at node n_1 would match the same activity name at node n_2 , producing wrong alignments.

New

Fix. Equality requires both $\text{act}(\ell_1) = \text{act}(\ell_2)$ and $\text{loc}(\ell_1) = \text{loc}(\ell_2)$. The hash function is $h(\ell) = \text{hash}(a, n)$.

Definition 3 (Located trace). *A located trace is a finite, temporally ordered sequence*

$$\sigma = \langle \ell_1^{t_1}, \ell_2^{t_2}, \dots, \ell_m^{t_m} \rangle, \quad \ell_i = (a_i, n_{j_i}) \in \mathcal{A} \times N, \quad t_1 < t_2 < \dots < t_m.$$

The local projection $\sigma|_n$ retains only events whose location is n .

Definition 4 (Event log). *An event log is a finite multiset $L = \{\sigma_1, \sigma_2, \dots, \sigma_p\}$ of located traces.*

1.2 Process Model

Definition 5 (Trie model). *A trie model is a rooted, labelled tree $\mathcal{T} = (Q, q_0, \delta, \lambda, F)$ where Q is a finite state set, $q_0 \in Q$ the root, $\delta \subseteq Q \times Q$ the parent-child relation, $\lambda: Q \setminus \{q_0\} \rightarrow \mathcal{A} \times N$ labels each non-root state with a located activity, and $F \subseteq Q$ is the set of accepting states.*

Every root-to-accepting-state path encodes one admissible located trace. We write $\mathcal{L}(\mathcal{T})$ for the language of $\mathcal{T}$.

Definition 6 (Trie label reachability). *For a subtree rooted at $q \in Q$, the reachable label set is*

$$R(q) = \{\lambda(q') \mid q' \in Q, q' \text{ is a descendant of } q\} \cup \{\lambda(q)\}.$$

We say a node q contains label ℓ iff $\ell \in R(q)$.

Old

Flaw: Recursive reachability check. Computed $R(q)$ via deep recursion: each call to `contains( $\ell$ )` triggered $O(|Q|)$ recursive descent, causing stack overflow on large tries.

New

Fix: Iterative BFS with caching. $R(q)$ is built once via iterative breadth-first traversal and cached. Subsequent containment queries are $O(1)$ set lookups. Children are additionally indexed by label in a hash map for $O(1)$ child access.

Definition 7 (Distributed trie model). *Given $\mathcal{T}$ over N , the distributed decomposition assigns to each $n_i \in N$ a local trie $\mathcal{T}_{n_i}^{\text{loc}}$. Every maximal contiguous subsequence of activities at n_i in any root-to-accepting-state path of $\mathcal{T}$ forms a branch in $\mathcal{T}_{n_i}^{\text{loc}}$.*

When a path transitions from n_j to n_i ($n_j(a') \rightarrow n_i(a)$, $n_j \neq n_i$), $n_j(a')$ is inserted as an entry point at the root of $\mathcal{T}_{n_i}^{\text{loc}}$; the subtree below it contains the local continuation at n_i .

Formally, $\mathcal{T}_{n_i}^{\text{loc}} = (Q_{n_i}, q_0^{n_i}, \delta_{n_i}, \lambda_{n_i}, F_{n_i})$ with root children partitioned as:

$$\begin{aligned} \text{localStarts}(n_i) &= \{q \mid \lambda_{n_i}(q) = n_i(a), a \text{ starts a trace at } n_i\}, \\ \text{entryPts}(n_i) &= \{q \mid \lambda_{n_i}(q) = n_j(a'), n_j \neq n_i, n_j(a') \text{ precedes a transition to } n_i\}, \\ \text{children}(q_0^{n_i}) &= \text{localStarts}(n_i) \cup \text{entryPts}(n_i). \end{aligned}$$

Definition 8 (Network). A network $\mathcal{G} = (N, \{\mathcal{T}_{n_i}^{\text{loc}}\}_{n_i \in N})$ pairs a node set with its local tries from Definition 7.

1.3 Alignment

Definition 9 (Alignment). *Given a located trace σ and a trie model $\mathcal{T}$, an alignment $\gamma = \langle m_1, \dots, m_p \rangle$ is a sequence of moves $(x_j^{\text{model}}, x_j^{\text{log}})$ of three types ($\ell = (a, n_i) \in \mathcal{A} \times N$):*

$$\begin{aligned} \textbf{Synchronous:} \quad & (\ell, \ell), \quad \text{cost} = 0, \\ \textbf{Model move:} \quad & (\ell, \gg), \quad \text{cost} = 1, \\ \textbf{Log move:} \quad & (\gg, \ell), \quad \text{cost} = 1. \end{aligned}$$

The alignment cost is $\text{cost}(\gamma) = \sum_{j=1}^p \text{cost}(m_j)$. An alignment is optimal if no other alignment for the same $(\sigma, \mathcal{T})$ has strictly lower cost.

Assumption (unit-cost). We assume $\text{cost}(\text{model move}) = \text{cost}(\text{log move}) = 1$ throughout.

Definition 10 (Alignment construction). *Given an alignment γ and a new move m , the extended alignment $\gamma' = \gamma \cdot m$ is created by appending m to the move sequence. The cost is tracked incrementally: $\text{cost}(\gamma') = \text{cost}(\gamma) + \text{cost}(m)$.*

Old

Flaw: Deep-copy construction. When using `deepcopy` on the entire alignment for every new move, for a trace of length m with branching factor b in A^* , this produced $O(b^m \cdot m)$ copied elements, causing exponential memory consumption.

New

Fix: Shallow-copy extension. Since alignment elements are immutable value objects, extending an alignment copies only the element *list* (not the elements themselves) and appends the new move. Cost is tracked as an integer field, not recomputed. This reduces per-move cost from $O(m)$ deepcopy to $O(1)$ amortised append.

Definition 11 (Covered events). *Given an alignment $\gamma = \langle m_1, \dots, m_p \rangle$, the set of covered events is*

$$\text{covered}(\gamma) = \{x_j^{\text{model}} \mid x_j^{\text{model}} \neq \gg\} \cup \{x_j^{\text{log}} \mid x_j^{\text{log}} \neq \gg\}.$$

Old

Flaw: One-sided coverage check. The original `append_missing_log_moves` only checked the *log side* of existing alignment elements: $\text{covered}_{\text{old}} = \{x_j^{\text{log}} \mid x_j^{\text{log}} \neq \gg\}$. This missed events already handled as synchronous or model moves, causing duplicate log moves in the final alignment.

New

Fix: Two-sided coverage. $\text{covered}(\gamma)$ includes events from *both* the model and log sides of every move (Definition 11). An external event is added as a log move only if it does not appear in $\text{covered}(\gamma)$.

Definition 12 (Central alignment). *Let $\mathcal{T}^c$ be a centralised trie built from training traces L_{train} . The central optimal alignment of a trace σ is*

$$\gamma_c^*(\sigma) = \arg \min_{\gamma \in \Gamma(\sigma, \mathcal{T}^c)} \text{cost}(\gamma),$$

where $\Gamma(\sigma, \mathcal{T}^c)$ is the set of all valid alignments of σ w.r.t. $\mathcal{T}^c$, computed via A^* over the product space (trie state $\times$ trace index).

2 Formal Problem Statement

Problem 1 (Distributed online conformance checking). *Given training traces L_{train} over $\mathcal{A}$ and $N = \{n_1, \dots, n_k\}$, construct a network $\mathcal{G} = (N, \{\mathcal{T}_{n_i}^{\text{loc}}\})$*

such that for every validation trace σ and every prefix $\sigma_{1..i}$, upon receiving the i -th event at node n_{j_i} , the system produces a distributed alignment γ_i with

$$\text{cost}(\gamma_i) = \text{cost}(\gamma_c^*(\sigma_{1..i})),$$

where γ_c^* is the optimal central alignment (Definition 12) using a centralised trie $\mathcal{T}^c$ built from the same L_{train} .

The computation must be distributed: each node n_i sees only its own local events $\sigma|_{n_i}$ and communicates with other nodes solely through the GETALIGN protocol (Definition 14).

The core requirement is *cost equivalence*: the distributed alignment must have the same cost as the centralised one. This demands that the decomposition into local tries preserves the structure of the global model, and that the composition of local alignments yields a globally optimal solution.

Definition 13 (Trace reconstruction). The reconstruction operator $\rho: \prod_{n_i} \mathcal{L}(\mathcal{T}_{n_i}^{\text{loc}}) \rightarrow (\mathcal{A} \times N)^*$ composes local fragments into full located traces by stitching at entry points: if $\mathcal{T}_{n_i}^{\text{loc}}$ has entry point e with $\lambda(e) = n_j(a')$, the local fragment at n_i is concatenated after the fragment at n_j ending with a' , respecting temporal order.

The decomposition must satisfy the language-preservation property:

$$\mathcal{L}(\mathcal{T}^c) = \rho\left(\prod_{n_i \in N} \mathcal{L}(\mathcal{T}_{n_i}^{\text{loc}})\right). \quad (1)$$

3 Mechanism: The GETALIGN Protocol

We now describe the distributed alignment protocol that solves Problem 1.

3.1 Protocol Definition

Definition 14 (GETALIGN protocol). Each node n_ℓ exposes $\text{GETALIGN}(n_\ell, e, t, V)$ returning an *AlignmentResponse* $(\gamma_{\text{up}}, t', e, n_{\text{last}})$ where:

- $e \in \text{entryPts}(n_\ell)$ is an entry-point label with $\text{loc}(e) = n_\ell$,
- t is the logical timestamp upper bound,
- $V \subseteq N$ is the visiting set (nodes already on the current call stack),
- γ_{up} is the best alignment of $\sigma|_{n_\ell}^{\leq t}$ against the subtrie reaching e ,
- t' is the timestamp of the last event consumed,
- n_{last} is the node that produced the deepest log moves.

Definition 15 (Cycle detection via visiting set). *Before node n_ℓ issues a recursive call $\text{GETALIGN}(n_k, e', t, V)$, it checks $n_k \notin V$. If $n_k \in V$, the entry point is skipped (the candidate set excludes this branch). Each call extends the set: $V' = V \cup \{n_\ell\}$.*

Old

Flaw: No cycle detection. The original protocol had no visiting set. If node A had an entry point labelled with a location at B , and B had an entry point labelled with a location at A , the mutual recursion $A \rightarrow B \rightarrow A \rightarrow B \rightarrow \dots$ never terminated, producing a stack overflow.

New

Fix. A set V is threaded through every recursive GETALIGN call. A node n is added to V before recursing; any call to a node already in V is pruned. This guarantees each node appears at most once on any call path.

3.2 Caching

Definition 16 (Response cache). *Each node n_i maintains a cache $C_{n_i}: (\mathcal{A} \times N) \times \mathbb{N} \rightarrow \text{AlignmentResponse}$ mapping (entry-point label, timestamp upper bound) pairs to previously computed responses.*

The cache is cleared at the start of each `process_event` call, ensuring no stale upstream results persist across events.

Old

Flaw: Stale cache. The original cache was keyed only by entry-point label, not by timestamp. Once a response was cached for entry point e , all subsequent events—at arbitrarily later times—reused the same stale response, ignoring new upstream events.

New

Fix. The cache key is the pair (e, t) . Additionally, the entire cache is invalidated at the start of each new event processing, so upstream results always reflect the current state of the network.

3.3 Distributed Online Alignment Procedure

Definition 17 (Distributed online alignment). *Given $\mathcal{G} = (N, \{\mathcal{T}_{n_i}^{\text{loc}}\})$ and a located trace $\sigma = \langle \ell_1^{t_1}, \dots, \ell_m^{t_m} \rangle$, upon receiving event $\ell_i = (a_i, n_{j_i})$ at time t_i , node n_{j_i} computes alignment γ_i for $\sigma_{1..i}$ as follows:*

1. **Entry-point enumeration.** Identify all children e of the root of $\mathcal{T}_{n_{j_i}}^{\text{loc}}$ whose subtree contains ℓ_i (i.e. $\ell_i \in R(e)$).
2. **Upstream alignment.** For each entry point e with $\text{loc}(e) = n_\ell \neq n_{j_i}$, request $\text{GETALIGN}(n_\ell, e, t_i, \{n_{j_i}\})$. If $n_\ell \in V$, skip this entry point (cycle avoidance).
3. **Local A^* alignment.** Compute a local alignment of the *full local trace* $\sigma|_{n_{j_i}}^{<t_i}$ against the subtree below e , targeting label ℓ_i .
4. **Composition.** Concatenate upstream and local alignments: $\gamma^{(e)} = \gamma_{\text{up}}(e) \cdot \gamma_{\text{loc}}(e)$.
5. **External log moves (top level only).** *Only the initiating node* — the node that received the original event — adds log moves for uncovered events at other nodes:

$$\gamma'^{(e)} = \gamma^{(e)} \cdot \langle (\gg, \ell) \mid \ell \in \bigcup_{n \neq n_{j_i}} \sigma|_n^{<t_i} \setminus \text{covered}(\gamma^{(e)}) \rangle.$$

6. **Selection.** Select the minimum-cost candidate over all entry points:

$$\gamma_i = \arg \min_{e \in \text{children}(q_0^{n_{j_i}})} \text{cost}(\gamma'^{(e)}).$$

Old

Flaw 1: External log moves added at every recursion level. The implementation called `_add_external_log_moves` inside every recursive `GETALIGN` call, not just at the top level. If node A added log moves for B 's events, and B added log moves for A 's events, the composed alignment contained duplicate log moves, inflating the cost.

New

Fix. External log moves are added *only* in `process_event` (the top-level entry point). Recursive `GETALIGN` calls return the upstream alignment without log-move patching. This ensures each external event is counted exactly once.

Old

Flaw 2: Local trace filtered by upstream timestamp. The original local trace retrieval was $\sigma|_{n_i}^{(t_{\text{up}}, t_i)}$ —only events *after* the upstream's last timestamp. If local events occurred before the upstream transition, they were silently dropped.

New

Fix. The local trace includes *all* events at the current node up to time t_i , regardless of the upstream timestamp: $\sigma|_{n_i}^{(-\infty, t_i)}$. The upstream timestamp informs only when the upstream alignment ends, not which local events to consider.

4 Proof of Soundness

We prove that the GETALIGN protocol produces optimal alignments via a series of lemmas.

4.1 Termination

Lemma 1 (Termination). *For any event $\ell_i^{t_i}$ arriving at node n_{j_i} , the recursive GETALIGN invocations terminate in at most $|N|$ steps.*

Proof. The visiting set V is initialised as $\{n_{j_i}\}$ and grows by one element at each recursive call (Definition 15). A recursive call $\text{GETALIGN}(n_k, e', t, V)$ is issued only if $n_k \notin V$. Since $|V| \leq |N|$ and V grows monotonically, the recursion depth is bounded by $|N| - 1$. At the base case, a node with only local starts ($\text{entryPts}(n) = \emptyset$) or all predecessors already in V computes a purely local A^* alignment without further recursion. $\square$

4.2 Cost Additivity

Lemma 2 (Cost additivity under composition). *Let γ_1 and γ_2 be alignments. Under unit costs,*

$$\text{cost}(\gamma_1 \cdot \gamma_2) = \text{cost}(\gamma_1) + \text{cost}(\gamma_2).$$

Proof. Each move's cost depends only on its type (synchronous = 0, model/log = 1). Concatenation appends the move sequences: $\gamma_1 \cdot \gamma_2 = \langle m_1^{(1)}, \dots, m_p^{(1)}, m_1^{(2)}, \dots, m_q^{(2)} \rangle$. The total cost is $\sum_{j=1}^p \text{cost}(m_j^{(1)}) + \sum_{j=1}^q \text{cost}(m_j^{(2)}) = \text{cost}(\gamma_1) + \text{cost}(\gamma_2)$. $\square$

4.3 Exhaustive Enumeration

Lemma 3 (Entry-point exhaustiveness). *Let σ^* be a path in $\mathcal{T}^c$ that passes through node n_i . Then there exists exactly one entry point $e \in \text{entryPts}(n_i) \cup \text{localStarts}(n_i)$ such that the local fragment of σ^* at n_i is a path in the subtree below e in $\mathcal{T}_{n_i}^{\text{loc}}$.*

Proof. By construction of the distributed decomposition (Definition 7):

- If σ^* starts at n_i , the first activity is a local start in $\text{localStarts}(n_i)$.

- If σ^* transitions from n_j to n_i via activity $n_j(a')$, then $n_j(a')$ is an entry point in $\text{entryPts}(n_i)$, and the continuation at n_i is the subtrie below this entry point.

Uniqueness: each transition point in a path determines exactly one entry point (or local start) in the local trie. Completeness: every transition from a path in $\mathcal{T}^c$ is represented as an entry point or local start in the corresponding local trie. $\square$

4.4 Local Optimality

Lemma 4 (A* local optimality). *Given a local trace $\sigma|_{n_i}$ and a subtrie $\mathcal{T}_{n_i}^{\text{loc}'}$ rooted below an entry point, the A* search over the product space $Q_{n_i} \times \{0, \dots, |\sigma|_{n_i}|\}$ returns an optimal local alignment.*

Proof. A* with an admissible heuristic (here $h = 0$, i.e. uniform-cost search) is complete and optimal on finite graphs. The product space is finite ($|Q_{n_i}| \cdot (|\sigma|_{n_i}| + 1)$ states), and the unit-cost function is non-negative. Therefore the first solution popped from the priority queue is optimal.

The goal check uses full located-activity equality (Definition 2), ensuring the search terminates at the correct target node in the trie. $\square$

Old

Flaw: Goal check ignored location. The original A* goal check compared only the activity name, not the full located activity. This could cause the search to terminate at a trie node with the right activity but wrong location, producing a spurious alignment.

4.5 Log-Move Completeness

Lemma 5 (External log-move completeness). *After step 5 of Definition 17, every event in $\sigma_{1..i}$ is covered by exactly one move in γ'_i .*

Proof. Partition the events in $\sigma_{1..i}$ by their location:

- Events at n_{j_i} (the current node): handled by the local A* alignment as synchronous or log moves.
- Events at upstream nodes along the chosen entry-point path: handled by the upstream GETALIGN alignment.
- Events at all other nodes: added as log moves in step 5, provided they are not already in $\text{covered}(\gamma^{(e)})$.

The coverage set covered (Definition 11) is constructed from *both* sides (model and log) of every existing element. Therefore, events already present

as synchronous moves (appearing on both sides) or model moves (appearing on the model side) are not duplicated. Events not in covered are added exactly once.

Step 5 is executed only at the top level (`process_event`), preventing nested addition across recursion levels. $\square$

4.6 Composition Optimality

Lemma 6 (Alignment composition optimality). *Let e be an entry point in $\mathcal{T}_{n_i}^{\text{loc}}$ with $\text{loc}(e) = n_\ell$. Let γ_{up}^* be optimal from $\text{GETALIGN}(n_\ell, e, t, V)$ and γ_{loc}^* optimal locally below e (Lemma 4). If the candidate set enumerates all $e \in \text{entryPts}(n_i) \cup \text{localStarts}(n_i)$ (Lemma 3), then*

$$\gamma_i = \arg \min_e (\text{cost}(\gamma_{\text{up}}^*(e)) + \text{cost}(\gamma_{\text{loc}}^*(e)))$$

is an optimal alignment for the events decomposable through n_i .

Proof. By Lemma 3, every decomposition of a global path through n_i corresponds to some entry point $e \in \text{entryPts}(n_i) \cup \text{localStarts}(n_i)$. For each e :

- The upstream alignment $\gamma_{\text{up}}^*(e)$ is optimal by the inductive hypothesis (or by Lemma 4 at the base case).
- The local alignment $\gamma_{\text{loc}}^*(e)$ is optimal by Lemma 4.
- By Lemma 2, $\text{cost}(\gamma_{\text{up}}^* \cdot \gamma_{\text{loc}}^*) = \text{cost}(\gamma_{\text{up}}^*) + \text{cost}(\gamma_{\text{loc}}^*)$.

Minimising over all entry points considers every possible decomposition, so the minimum is globally optimal among all decompositions through n_i . $\square$

4.7 Main Theorem

Theorem 1 (Soundness of distributed online alignment). *Under the unit-cost assumption and with the corrected protocol (Definitions 14–17), for every validation trace σ and every prefix $\sigma_{1..i}$, the distributed alignment γ_i satisfies*

$$\text{cost}(\gamma_i) \leq \text{cost}(\gamma_c^*(\sigma_{1..i})).$$

Proof. By induction on the recursion depth of `GETALIGN`.

Base case (depth 0). Node n_i has only local starts ($\text{entryPts}(n_i) = \emptyset$) or all predecessor nodes are in V . The alignment is computed by A^* on the local product space, which is optimal by Lemma 4.

Inductive step (depth d). Assume `GETALIGN` returns optimal sub-alignments at depth $\leq d - 1$. At depth d , node n_i :

1. Enumerates all entry points — exhaustive by Lemma 3.

2. For each entry point, obtains an optimal upstream alignment (by the inductive hypothesis).
3. Computes an optimal local alignment (Lemma 4).
4. Composes them with additive cost (Lemma 2).
5. Adds external log moves exactly once, covering all remaining events (Lemma 5).
6. Selects the minimum over all candidates (Lemma 6).

The recursion terminates by Lemma 1. The resulting γ_i has cost no greater than any centralised alignment, since every decomposition of a centralised path is represented in the candidate set. $\square$

Corollary 1. *The protocol terminates in at most $|N|$ recursive steps per event, and each invocation explores a local state space of size $|Q_{n_i}| \cdot (|\sigma|_{n_i} + 1)$, which is substantially smaller than the centralised state space $|Q_c| \cdot (m + 1)$ when activities are distributed across nodes.*

5 State Space Complexity

Definition 18 (Alignment state space). *A* alignment of a length- m trace against a trie with $|Q|$ states explores the product space $S = Q \times \{0, 1, \dots, m\}$, of size $|S| = |Q| \cdot (m + 1)$.*

Proposition 1 (Centralised state space). *For the centralised trie $\mathcal{T}^c$, in the worst case $|Q_c| = O(\sum_{\sigma \in L_{\text{train}}} |\sigma|)$ (no shared prefixes), giving search space*

$$|S_c| = |Q_c| \cdot (m + 1) = O\left(\left(\sum_{\sigma \in L_{\text{train}}} |\sigma|\right) \cdot m\right).$$

Proposition 2 (Distributed state space). *Each node n_i searches $S_{n_i} = Q_{n_i} \times \{0, \dots, |\sigma|_{n_i}|\}$. The total effort is bounded by*

$$\sum_{n_i \in N} |Q_{n_i}| \cdot (|\sigma|_{n_i} + 1).$$

Since both trie states and trace are partitioned, $|Q_{n_i}| \ll |Q_c|$ and $|\sigma|_{n_i} \leq m$, so each local product is substantially smaller than $|Q_c| \cdot m$.

Cascading overhead. The worst case produces a chain of $|N|$ sequential GETALIGN calls, each blocking on its predecessor, yielding $O(|N|)$ sequential A* invocations. Caching (Definition 16) reduces repeated requests to amortised $O(1)$.

Entry-point enumeration. If n_i has $E_i = |\text{entryPts}(n_i)|$ entry points, the worst-case per-event effort is $O(E_i \cdot (|Q_{n_i}| \cdot |\sigma|_{n_i} + C_{\text{up}}))$, where C_{up} is the upstream cascade cost.

In summary, the decomposition trades the monolithic space $|Q_c| \cdot m$ for a sum of smaller local spaces at the cost of inter-node communication. It is most effective when (i) $|N|$ is moderate, (ii) activities are well-distributed ($|Q_{n_i}| \ll |Q_c|$), and (iii) the number of entry points E_i per node is bounded.